

# Software Requirements Specification (SRS)

## G13 Project - Role Management System

### 1. Introduction

#### 1.1 Purpose

This document specifies the software requirements for the **Role Management System**, developed by SWE Project Group 13. It defines the system architecture, core features, functional requirements (organized by Epic), and non-functional requirements to guide ongoing maintenance, testing, and future development.

#### 1.2 Scope

The system is a centralized RBAC platform designed to define roles, permissions, resource scoping, and dynamic policy evaluation across multiple independent microservices and React-based microfrontends. It serves both backend API authorization and frontend UI component access gating (Microfrontend scoping). The system features an Authorization Engine (auz-engine), dedicated management services, and administrator portals.

#### 1.3 Definitions and Acronyms

- **RBAC:** Role-Based Access Control
- **ABAC:** Attribute-Based Access Control
- **MFE:** Microfrontend
- **JWT:** JSON Web Token
- **API:** Application Programming Interface
- **MoSCoW:** Prioritization technique (Must have, Should have, Could/Nice to have, Won't have)

### 2. System Overview & Architecture

#### 2.1 Product Perspective & Tech Stack

The platform is a distributed system acting as the central authorization hub for organizational microservices.

- **Backend Services:** Node.js ( $\geq 18$ ) and Express
- **Frontend Applications:** React 19, Vite, Module Federation
- **Database:** MongoDB (Atlas or local instance)

## 2.2 System Components

- **auz-engine (Port 3000):** The core authorization engine. Issues JWTs, resolves access profiles, and evaluates permissions.
- **registry-service (Port 5001):** Manages the catalog of API endpoints and MFE modules.
- **role-service (Port 3002):** Handles CRUD operations for roles, permissions, and temporary delegations.
- **user-service (Port 3003):** Manages user identities and role assignments.
- **Admin-Registry-Management (MFE - Port 5002):** UI portal for managing registered APIs and MFEs.
- **Admin-Role-Creation (MFE - Port 5173):** UI portal for designing roles and assigning permissions.

## 2.3 User Classes

1. **Super Admins / Platform Admins:** Full access to create roles, modify registries, and manage the platform.
2. **Service Administrators:** Manage specific subsets of resources (e.g., Finance Manager).
3. **End Users:** Consume applications granted via assigned permanent or temporary roles.
4. **Temporary Users:** Have time-bound access roles that automatically expire (e.g., Contractors, Auditors).

# 3. Core System Mechanisms

## 3.1 Permission Sets & Backend/Frontend Mapping

The system utilizes unified Permission Sets that bridge backend API access with frontend MFE visibility. This is achieved through:

- **Permission Sets:** Logical groupings of discrete permissions (service:resource:action) assigned to roles. These sets reduce payload bloat and simplify role assignment.
- **Frontend Mapping:** MFE route configurations map directly to these permission sets. When the MFE shell loads, it queries the auz-engine to retrieve the hydrated MFE objects and feature slugs permitted for the active session, dynamically building the navigation tree and routing rules.

## 3.2 State Management & Edge Cases

- **Graceful 403 Handling:** MFE wrappers must intercept 403 Forbidden responses natively. Rather than crashing the broader MFE shell application, they render a localized fallback UI (e.g., an "Access Denied" or "Unauthorized Component" view).
- **Mid-Session Revocation:** If a user's permissions are revoked or a temporary role expires while active, the frontend state must forcefully flush the cached permissions via WebSocket invalidation or short-polling, redirecting the user or re-rendering the shell to reflect the updated state.

## 4. Functional Requirements (Epics)

The system's functional capabilities are categorized into four primary Epics, prioritized using the MoSCoW method.

### 4.1 Epic 1: Authorization Engine (Core Backend)

*The critical path. This engine sits in the middle of every request and must be highly performant.*

| Feature                       | Priority  | Description                                                                                    |
|-------------------------------|-----------|------------------------------------------------------------------------------------------------|
| <b>Request Context Parser</b> | Must Have | Extract user identity, resource, action (resource.action), and environment attributes.         |
| <b>RBAC Evaluator</b>         | Must Have | Retrieve/expand roles associated with the user ID and match against requested resource.action. |
| <b>ABAC Evaluator</b>         | Must Have | Fetch resource/user attributes and parse/execute conditional logic rules.                      |
| <b>Conflict Resolver</b>      | Must Have | Combine RBAC and ABAC results based on a strict evaluation strategy.                           |

| Feature                      | Priority    | Description                                                           |
|------------------------------|-------------|-----------------------------------------------------------------------|
| <b>Standardized Response</b> | Must Have   | Return a unified JSON payload ({ allowed: boolean, reason: string }). |
| <b>Decision Caching</b>      | Should Have | Use Redis to cache identical RBAC decisions for a short TTL.          |

## 4.2 Epic 2: Admin Management (MFE + API)

*Where security administrators govern the system, manage roles, and handle temporary access.*

| Feature                        | Priority    | Description                                                                                 |
|--------------------------------|-------------|---------------------------------------------------------------------------------------------|
| <b>Role CRUD &amp; Mapping</b> | Must Have   | UI/APIs to manage roles and checkbox UI to attach/detach specific permission sets.          |
| <b>Permission Catalog</b>      | Must Have   | View all available resource.action permissions synced from the API Registry.                |
| <b>Temporary Access</b>        | Must Have   | valid_until timestamps for assignments and a UI form for users to request temporary access. |
| <b>Automated Cleanup</b>       | Should Have | Node-cron job running hourly to physically delete expired role assignments.                 |

| Feature           | Priority    | Description                                                       |
|-------------------|-------------|-------------------------------------------------------------------|
| Approval Workflow | Should Have | Access requests sit in a "Pending" state until an Admin approves. |

### 4.3 Epic 3: Developer Portal & Integration (MFE + API)

*Where developers onboard their applications, register APIs, and manage integrations.*

| Feature              | Priority  | Description                                                                  |
|----------------------|-----------|------------------------------------------------------------------------------|
| Service Registration | Must Have | Create new app profiles (Name, Owner, Base URL) and define domain resources. |
| Endpoint Config      | Must Have | Map HTTP methods and routes to specific actions.                             |
| MFE Route Protection | Must Have | Wrapper component querying auz-engine before lazy-loading a module.          |

### 4.4 Epic 4: Audit & Observability System

| Feature             | Priority  | Description                                    |
|---------------------|-----------|------------------------------------------------|
| Standard Log Schema | Must Have | Define a strict JSON structure for all events. |
| Decision Logging    | Must Have | Log every Allow/Deny outcome generated by the  |

| Feature            | Priority    | Description                                                                       |
|--------------------|-------------|-----------------------------------------------------------------------------------|
|                    |             | Authorization Engine.                                                             |
| Audit Dashboard UI | Should Have | Data table in Admin portal to view logs, filterable by Date, User ID, and Status. |

## 5. Non-Functional Requirements

### 5.1 Performance

**O(1) Lookups:** Permission verification (requirePermission) must execute in O(1) time utilizing flat, in-memory data structures. The auz-engine must efficiently merge role permissions on initial load and utilize caching to minimize database hits on subsequent requests.

### 5.2 Security

**Fail-Secure:** The system must silently deny access (403 Forbidden) if a permission key is missing, unrecognized, or if a temporary role has expired. All backend services must secure sensitive internal endpoints using a shared JWT\_SECRET.

### 5.3 Reliability & Maintainability

**Decoupled Architecture:** Services must operate independently with distinct database connections and environment variables to prevent cascading failures.

## 6. Future Enhancements (Beyond Scope)

- **Advanced Access Simulator:** An administrative UI tool to thoroughly test if a specific userId has a specific permissionKey across complex inherited roles.
- **Comprehensive Audit Append-Log:** A strictly append-only, immutable log capturing all permission changes and access failures for high-level compliance requirements.